

BVMT AI Platform — TFT Data Quality Report

pgAdmin SQL Queries + Jupyter Notebook — Run before training

Esprit School of Business — Tunisia | March 2026

Why check data quality before training?

Training a TFT model on dirty data is worse than not training at all. Garbage in = garbage out. If your price data has impossible values (like close price higher than the day's high), the model will learn the wrong patterns and give you completely wrong predictions. These checks take 20 minutes and can save you 2 days of debugging.

The 3 tables that feed TFT:

- `daily_prices` — the main OHLCV data: open, close, low, high, volume for each stock each day
- `computed_indicators` — MA5, MA20, MA50, RSI14, volatility, `daily_return` derived from prices
- `company_metadata` — static info: ticker, `isin_code`, `groupe` (used to classify bank/non_bank)

You have 9 checks to run. Run ALL of them in pgAdmin first (fast, no Python needed). Then run the Jupyter notebook for the visual charts. Only proceed to `feature_engineering.ipynb` when all 9 show PASS.

Part 1 — pgAdmin SQL Queries

Open pgAdmin → bvmc_db → Query Tool. Run each query separately. Copy the query, paste it, press F5. Read the result and compare to the expected output described below each query.

CHECK 1 — Row counts and date range

WHY: Confirm you have the expected amount of data. If daily_prices has fewer than 100,000 rows, something went wrong during loading.

```
SELECT
    'daily_prices'          AS table_name,
    COUNT(*)                AS total_rows,
    COUNT(DISTINCT ticker)  AS unique_stocks,
    COUNT(DISTINCT seance)  AS unique_dates,
    MIN(seance)             AS earliest_date,
    MAX(seance)             AS latest_date
FROM daily_prices

UNION ALL

SELECT
    'computed_indicators' AS table_name,
    COUNT(*)              AS total_rows,
    COUNT(DISTINCT isin_code) AS unique_stocks,
    COUNT(DISTINCT seance)  AS unique_dates,
    MIN(seance)             AS earliest_date,
    MAX(seance)             AS latest_date
FROM computed_indicators;
```

EXPECTED RESULTS:

daily_prices → ~120,000+ rows | 60-68 stocks | ~2,000 unique dates | 2016 to 2025
computed_ind → ~115,000+ rows | same stocks | dates start ~50 days later (MA50 warmup)

If daily_prices has fewer than 80,000 rows → some CSV files may not have loaded
If computed_indicators is empty → run compute_indicators.ipynb again

CHECK 2 — NULL values in every column

WHY: NULL values crash the TFT training. Even one NULL in close_price will cause an error. We need to know exactly which columns have NULLs and how many.

```
-- Check NULLs in daily_prices
SELECT
    COUNT(*) FILTER (WHERE seance IS NULL)          AS null_seance,
    COUNT(*) FILTER (WHERE isin_code IS NULL)        AS null_isin,
```

```

COUNT(*) FILTER (WHERE ticker IS NULL)           AS null_ticker,
COUNT(*) FILTER (WHERE ouverture IS NULL)        AS null_open,
COUNT(*) FILTER (WHERE cloture IS NULL)           AS null_close,
COUNT(*) FILTER (WHERE plus_bas IS NULL)          AS null_low,
COUNT(*) FILTER (WHERE plus_haut IS NULL)         AS null_high,
COUNT(*) FILTER (WHERE quantite_negociee IS NULL) AS null_volume,
COUNT(*) FILTER (WHERE nb_transaction IS NULL)    AS null_trades
FROM daily_prices;

```

```

-- Check NULLs in computed_indicators
SELECT
COUNT(*) FILTER (WHERE ma_5 IS NULL)           AS null_ma5,
COUNT(*) FILTER (WHERE ma_20 IS NULL)          AS null_ma20,
COUNT(*) FILTER (WHERE ma_50 IS NULL)          AS null_ma50,
COUNT(*) FILTER (WHERE rsi_14 IS NULL)         AS null_rsi,
COUNT(*) FILTER (WHERE daily_return IS NULL)   AS null_return,
COUNT(*) FILTER (WHERE volatility_20 IS NULL)   AS null_vol
FROM computed_indicators;

```

WHAT TO EXPECT:

daily_prices: ALL null counts should be 0. If null_close > 0 → data loading problem.

computed_indicators: null_ma50 will be > 0 (normal — MA50 needs 50 days of warmup).

null_ma50 ≈ number_of_stocks × 50 rows ≈ 3,000-4,000 is perfectly fine.

null_rsi will also be > 0 for the same reason (RSI needs 14+ days).

IF null_close > 0 in daily_prices: run this to find which stocks have the problem:

```
SELECT ticker, COUNT(*) FROM daily_prices WHERE cloture IS NULL GROUP BY ticker;
```

CHECK 3 — Price logic — impossible combinations

WHY: In any financial market, the HIGH price of the day must be greater than or equal to both the LOW and the CLOSE. If high < low, that is physically impossible — it means a data error. TFT will learn the wrong price dynamics if these errors exist.

Simple example: If AMEN BANK on January 5 shows open=12.5, close=13.8, low=14.0, high=11.0 — that makes no sense. The low cannot be higher than the close and the high cannot be lower than the low.

```

-- Check 1: HIGH must be >= LOW (most basic rule)
SELECT ticker, seance, plus_bas AS low, plus_haut AS high, cloture AS close
FROM daily_prices
WHERE plus_haut < plus_bas
ORDER BY seance
LIMIT 20;

```

```

-- Check 2: CLOSE must be between LOW and HIGH
SELECT ticker, seance,
       plus_bas AS low, cloture AS close, plus_haut AS high
FROM daily_prices

```

```

WHERE cloture > plus_haut * 1.001 -- close above high (with 0.1% tolerance)
   OR cloture < plus_bas * 0.999 -- close below low (with 0.1% tolerance)
ORDER BY seance
LIMIT 20;

```

```

-- Check 3: OPEN must be between LOW and HIGH
SELECT ticker, seance,
       plus_bas AS low, ouverture AS open, plus_haut AS high
FROM daily_prices
WHERE ouverture > plus_haut * 1.001
   OR ouverture < plus_bas * 0.999
ORDER BY seance
LIMIT 20;

```

```

-- Check 4: All prices must be positive
SELECT ticker, seance, ouverture, cloture, plus_bas, plus_haut
FROM daily_prices
WHERE cloture <= 0 OR plus_haut <= 0 OR plus_bas <= 0
ORDER BY seance
LIMIT 20;

```

PASS CONDITION: All 4 queries should return 0 rows.

If any query returns rows → those are data errors.

Fix option A: `DELETE FROM daily_prices WHERE [error condition]` — remove bad rows.

Fix option B: `UPDATE daily_prices SET cloture = (plus_bas + plus_haut) / 2`
`WHERE cloture > plus_haut` — estimate close as midpoint.

For TFT training, Fix A is safer — fewer rows is better than wrong rows.

CHECK 4 — Volume logic

WHY: Volume (number of shares traded) cannot be negative. Zero volume means the stock was suspended that day — this is normal for BVMT. But we need to know the percentage of zero-volume days because too many zeros reduces model quality.

```

-- Check for negative volumes (should never happen)
SELECT ticker, seance, quantite_negocie AS volume
FROM daily_prices
WHERE quantite_negocie < 0
LIMIT 10;

```

```

-- Count zero-volume days per stock and as percentage
SELECT
  ticker,
  COUNT(*) AS total_days,
  COUNT(*) FILTER (WHERE quantite_negocie = 0) AS zero_volume_days,
  ROUND(
    COUNT(*) FILTER (WHERE quantite_negocie = 0)::numeric
    / COUNT(*) * 100, 1
  ) AS zero_pct

```

```
FROM daily_prices
GROUP BY ticker
ORDER BY zero_pct DESC
LIMIT 20;
```

EXPECTED: No negative volumes. Zero volumes are normal.

Acceptable: up to 15% zero-volume days per stock (BVMT has many illiquid stocks).

Warning: if a stock has >30% zero-volume days, it is too illiquid for TFT training.

→ Consider removing very illiquid stocks from training using the 500-day filter in feature_engineering.ipynb Cell 7.

Why zero volume matters: on zero-volume days, the price often stays exactly the same as the day before (no trading = no price discovery). This creates flat periods in the time series that confuse the model. Not a dealbreaker but worth knowing.

CHECK 5 — Extreme price movements (data errors vs real events)

WHY: A stock moving +200% in one day is almost certainly a data error, not a real event. BVMT has a daily price limit of approximately +/-6.1% for most stocks. Moves much larger than this suggest a wrong data entry.

Important exception: Rights issues (augmentations de capital) can cause apparent large drops because the ex-rights price can be 20-30% lower than the cum-rights price overnight. These are real events, not errors.

```
-- Find extreme single-day price movements
WITH daily_moves AS (
  SELECT
    ticker,
    seance,
    cloture AS close_today,
    LAG(cloture) OVER (
      PARTITION BY ticker ORDER BY seance
    ) AS close_yesterday,
    ROUND(
      (cloture - LAG(cloture) OVER (PARTITION BY ticker ORDER BY seance))
      / NULLIF(LAG(cloture) OVER (PARTITION BY ticker ORDER BY seance), 0)
      * 100, 2
    ) AS pct_change
  FROM daily_prices
)
SELECT ticker, seance, close_yesterday, close_today, pct_change
FROM daily_moves
WHERE ABS(pct_change) > 20 -- flag moves larger than 20% in one day
ORDER BY ABS(pct_change) DESC
LIMIT 30;
```

HOW TO READ THE RESULTS:

pct_change = +25%: price went up 25% in one day

pct_change = -30%: price fell 30% in one day

Investigate each flagged row. Ask: did this company have a capital event on that date?

If yes (rights issue, stock split, dividend) → the data may be correct.

If no obvious reason → likely a data error. Either delete or correct that row.

For TFT: rows with $|\text{pct_change}| > 50\%$ should almost always be deleted.

The model will overfit to these extreme events and perform worse overall.

CHECK 6 — Coverage per stock — do you have enough history?

WHY: TFT needs a minimum of 60 consecutive trading days just to make one prediction (the encoder window). Stocks with very little history cannot contribute meaningfully to training. We set 500 days (~2 years) as the minimum.

```
-- Coverage per stock: trading days, date range, and years of history
SELECT
  ticker,
  COUNT(*) AS trading_days,
  MIN(seance) AS first_date,
  MAX(seance) AS last_date,
  ROUND(
    EXTRACT(EPOCH FROM (MAX(seance) - MIN(seance)))
    / 86400 / 365.25, 1
  ) AS years_of_history,
  CASE
    WHEN COUNT(*) >= 1000 THEN 'EXCELLENT (8+ years)'
    WHEN COUNT(*) >= 500  THEN 'GOOD (2-8 years)'
    WHEN COUNT(*) >= 252  THEN 'MINIMAL (1-2 years)'
    ELSE                      'TOO FEW - exclude from TFT'
  END AS status
FROM daily_prices
GROUP BY ticker
ORDER BY trading_days DESC;
```

TARGET: All 15 key stocks should show EXCELLENT or GOOD status.

Key stocks that MUST have 1000+ days: AMEN BANK, BIAT, BNA, STB, ATTIJARI BANK, ATB, SFBT, POULINA, ARTES, CARTHAGE CEMENT, UIB, UBCI, BH, ONE TECH, DELICE.

Stocks with < 500 days will be automatically filtered in feature_engineering.ipynb.

This is not a problem — it is expected for stocks listed after 2022.

CHECK 7 — Gaps in trading dates — missing days

WHY: TFT works with a continuous time index. If AMEN BANK has data for Monday and Thursday but nothing for Tuesday and Wednesday, that creates a gap. Gaps do not necessarily break TFT (it uses time_idx integers not actual dates) but large gaps indicate possible missing CSV files.

```
-- Count trading days per year per stock
-- BVMT typically has 240-252 trading days per year
SELECT
    ticker,
    EXTRACT(YEAR FROM seance)::int AS year,
    COUNT(*) AS trading_days
FROM daily_prices
GROUP BY ticker, year
ORDER BY ticker, year;
```

```
-- Find years with suspiciously few trading days
-- (might mean a CSV file for that year did not load)
SELECT
    ticker,
    EXTRACT(YEAR FROM seance)::int AS year,
    COUNT(*) AS trading_days,
    CASE
        WHEN COUNT(*) < 100 THEN 'WARNING - possible missing data'
        WHEN COUNT(*) < 200 THEN 'LOW - check if stock was listed this year'
        ELSE 'OK'
    END AS status
FROM daily_prices
GROUP BY ticker, year
HAVING COUNT(*) < 200
ORDER BY ticker, year;
```

WHAT IS NORMAL:

2016-2025: expect 240-252 trading days per year per stock (BVMT closes weekends + holidays).

First year a stock is listed: fewer days (stock may have listed mid-year) — normal.

Last year (2025): fewer days depending on when you downloaded the data — normal.

WARNING: If a mid-career year like 2019 or 2021 shows <150 days for a major bank like AMEN BANK → your 2019.txt or 2021.txt file may not have loaded correctly.

Go back to load_prices.ipynb and recheck.

CHECK 8 — Indicator range validation

WHY: RSI must always be between 0 and 100 — that is a mathematical property of the RSI formula. If you see RSI = 150 or RSI = -5, your indicator computation has a bug. Moving averages must be positive (they are averages of positive prices). Volatility must be positive.

```
-- Validate RSI is in [0, 100]
SELECT
  COUNT(*) FILTER (WHERE rsi_14 < 0) AS rsi_below_zero,
  COUNT(*) FILTER (WHERE rsi_14 > 100) AS rsi_above_100,
  MIN(rsi_14) AS rsi_min,
  MAX(rsi_14) AS rsi_max,
  ROUND(AVG(rsi_14)::numeric, 2) AS rsi_avg
FROM computed_indicators
WHERE rsi_14 IS NOT NULL;
```

```
-- Validate moving averages are positive and sensible
SELECT
  COUNT(*) FILTER (WHERE ma_5 <= 0) AS bad_ma5,
  COUNT(*) FILTER (WHERE ma_20 <= 0) AS bad_ma20,
  COUNT(*) FILTER (WHERE ma_50 <= 0) AS bad_ma50,
  ROUND(MIN(ma_20)::numeric, 3) AS ma20_min,
  ROUND(MAX(ma_20)::numeric, 3) AS ma20_max,
  ROUND(AVG(ma_20)::numeric, 3) AS ma20_avg
FROM computed_indicators
WHERE ma_20 IS NOT NULL;
```

```
-- Validate daily_return is reasonable
-- Returns > 100% per day are almost certainly errors
SELECT
  COUNT(*) FILTER (WHERE daily_return > 1.0) AS return_above_100pct,
  COUNT(*) FILTER (WHERE daily_return < -1.0) AS return_below_neg100pct,
  ROUND(MIN(daily_return)::numeric*100, 2) AS min_return_pct,
  ROUND(MAX(daily_return)::numeric*100, 2) AS max_return_pct,
  ROUND(AVG(daily_return)::numeric*100, 4) AS avg_return_pct
FROM computed_indicators
WHERE daily_return IS NOT NULL;
```

PASS CONDITIONS:

```
rsi_below_zero = 0 AND rsi_above_100 = 0
bad_ma5 = 0 AND bad_ma20 = 0 AND bad_ma50 = 0
return_above_100pct = 0 AND return_below_neg100pct = 0
```

RSI average should be near 50 (slightly above is normal in a bull market).
 MA20 average should be similar to the average close price in daily_prices.
 Average daily return should be slightly positive (e.g. 0.02% per day).

CHECK 9 — Join test — how many rows survive?

WHY: feature_engineering.ipynb joins daily_prices with computed_indicators on (seance, isin_code). If the join keys do not match (e.g. one table uses ISIN codes with leading zeros and the other does not), you lose data silently. This check confirms the join works before you build the full dataset.

```
-- Test the join and count surviving rows
```



```

SELECT
    COUNT(*)                AS joined_rows,
    COUNT(DISTINCT dp.ticker) AS stocks_in_join,
    MIN(dp.seance)           AS earliest_joined_date,
    MAX(dp.seance)           AS latest_joined_date
FROM daily_prices dp
INNER JOIN computed_indicators ci
    ON dp.seance = ci.seance
   AND dp.isin_code = ci.isin_code;

-- Check for isin_code mismatches between tables
-- Any rows here mean those ISINs exist in one table but not the other
SELECT 'in prices but not indicators' AS issue, isin_code
FROM (SELECT DISTINCT isin_code FROM daily_prices) p
WHERE isin_code NOT IN (SELECT DISTINCT isin_code FROM computed_indicators)

UNION ALL

SELECT 'in indicators but not prices' AS issue, isin_code
FROM (SELECT DISTINCT isin_code FROM computed_indicators) i
WHERE isin_code NOT IN (SELECT DISTINCT isin_code FROM daily_prices);

```

PASS CONDITIONS:

joined_rows should be > 110,000 (slightly less than daily_prices due to MA warmup).

stocks_in_join should equal the number of stocks in daily_prices.

The UNION ALL query should return 0 rows (all ISINs match).

If the second query returns rows: there is an ISIN format mismatch.

Fix: UPDATE one table so the formats match, e.g.:

```
UPDATE computed_indicators SET isin_code = TRIM(isin_code);
```

```
UPDATE daily_prices SET isin_code = UPPER(isin_code);
```

Part 2 — Jupyter Notebook: check_data_quality.ipynb

Create notebooks/check_data_quality.ipynb. Copy each section below into a separate cell. Run all cells in order. The notebook gives you visual charts that pgAdmin cannot show — histograms, coverage heatmaps, and distribution plots that make problems obvious at a glance.

Cell 1 — Imports and data loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import psycopg2
import os
import warnings
warnings.filterwarnings('ignore')
from dotenv import load_dotenv
load_dotenv()

def get_conn():
    return psycopg2.connect(
        host=os.getenv('DB_HOST'), port=int(os.getenv('DB_PORT', 5432)),
        dbname=os.getenv('DB_NAME'), user=os.getenv('DB_USER'),
        password=os.getenv('DB_PASSWORD'))

conn = get_conn()

# Load both tables
df = pd.read_sql('''
    SELECT seance, isin_code, ticker,
           ouverture AS open_p, cloture AS close_p,
           plus_bas AS low_p, plus_haut AS high_p,
           quantite_negocie AS volume
    FROM daily_prices ORDER BY ticker, seance
''', conn)

df_ind = pd.read_sql('''
    SELECT seance, isin_code, ma_5, ma_20, ma_50,
           rsi_14, daily_return, volatility_20
    FROM computed_indicators ORDER BY isin_code, seance
''', conn)
conn.close()

df['seance'] = pd.to_datetime(df['seance'])
df_ind['seance'] = pd.to_datetime(df_ind['seance'])

print(f'daily_prices      : {len(df):,} rows | {df["ticker"].nunique()} stocks')
print(f'indicators        : {len(df_ind):,} rows')
print(f'Date range          : {df["seance"].min().date()} to {df["seance"].max().date()}')
```

Cell 2 — Null check (visual)

This cell shows you exactly how many NULLs exist in each column, as a bar chart.

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# daily_prices nulls
price_cols = ['open_p', 'close_p', 'low_p', 'high_p', 'volume']
price_nulls = df[price_cols].isnull().sum()
color_p = ['#d32f2f' if n > 0 else '#388e3c' for n in price_nulls]
axes[0].bar(price_cols, price_nulls, color=color_p)
axes[0].set_title('NULLs in daily_prices', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Number of NULL rows')
for i, v in enumerate(price_nulls):
    axes[0].text(i, v + 10, str(v), ha='center', fontsize=10,
                 color='red' if v > 0 else 'green')

# indicators nulls
ind_cols = ['ma_5', 'ma_20', 'ma_50', 'rsi_14', 'daily_return', 'volatility_20']
ind_nulls = df[ind_cols].isnull().sum()
expected = df['ticker'].nunique() * 55 # ~55 warmup rows per stock
color_i = ['#f57c00' if n > expected*2 else '#388e3c' for n in ind_nulls]
axes[1].bar(ind_cols, ind_nulls, color=color_i)
axes[1].set_title('NULLs in computed_indicators', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Number of NULL rows')
axes[1].axhline(y=expected, color='orange', linestyle='--',
                 label=f'Expected warmup NULLs (~{expected:,})')
axes[1].legend(fontsize=9)
for i, v in enumerate(ind_nulls):
    axes[1].text(i, v + 50, f'{v:,}', ha='center', fontsize=9)

plt.tight_layout()
plt.savefig('data_quality_nulls.png', dpi=120, bbox_inches='tight')
plt.show()

# Summary
if price_nulls.sum() == 0:
    print('PASS: No NULLs in daily_prices')
else:
    print(f'FAIL: {price_nulls.sum()} NULLs in daily_prices - see red bars')

```

Cell 3 — Price logic check (automatic)

This cell checks all 4 price logic rules and prints a clear PASS or FAIL for each one.

```

checks = {}

df_c = df.dropna(subset=['open_p', 'close_p', 'low_p', 'high_p'])

# Rule 1: high >= low
bad_hl = df_c[df_c['high_p'] < df_c['low_p']]
checks['high >= low'] = len(bad_hl) == 0

# Rule 2: close between low and high
bad_cl = df_c[(df_c['close_p'] > df_c['high_p']*1.001) |
               (df_c['close_p'] < df_c['low_p']*0.999)]
checks['close in [low, high]'] = len(bad_cl) <= 50

# Rule 3: all prices positive
bad_pos = df_c[df_c['close_p'] <= 0]
checks['close > 0'] = len(bad_pos) == 0

```

```

# Rule 4: no negative volume
bad_vol = df_c[df_c['volume'] < 0]
checks['volume >= 0'] = len(bad_vol) == 0

# Rule 5: extreme moves
df_sorted = df_c.sort_values(['ticker', 'seance'])
df_sorted['prev'] = df_sorted.groupby('ticker')['close_p'].shift(1)
df_sorted['chg'] = (df_sorted['close_p'] - df_sorted['prev']) / df_sorted['prev']
extreme = df_sorted[df_sorted['chg'].abs() > 0.60].dropna(subset=['chg'])
checks['moves < 60%/day'] = len(extreme) < 20

print('PRICE LOGIC CHECKS:')
print('-' * 40)
all_pass = True
for name, passed in checks.items():
    icon = 'PASS' if passed else 'FAIL'
    print(f' {icon} {name}')
    if not passed: all_pass = False

if not all_pass and len(bad_hl) > 0:
    print(f'\nSample of high < low errors:')
    print(bad_hl[['ticker', 'seance', 'low_p', 'high_p']].head(5).to_string())

if not all_pass and len(extreme) > 0:
    print(f'\nExtreme moves (>60%/day):')
    print(extreme[['ticker', 'seance', 'prev', 'close_p', 'chg']].head(10).to_string())

```

Cell 4 — Coverage heatmap (stocks × years)

This chart shows you at a glance which stocks have data for which years. Each cell is colored green (good coverage), yellow (partial), or red (missing). This is the fastest way to spot a missing CSV year.

```

df['year'] = df['seance'].dt.year
coverage = df.groupby(['ticker', 'year'])['seance'].count().unstack(fill_value=0)

# Calculate expected trading days (roughly 252 per year)
cov_pct = coverage / 252 * 100
cov_pct = cov_pct.clip(0, 100)

fig, ax = plt.subplots(figsize=(15, max(8, len(coverage)*0.3)))

cmap = mcolors.LinearSegmentedColormap.from_list(
    'traffic', ['#d32f2f', '#f57c00', '#fdd835', '#66bb6a', '#2e7d32'])

im = ax.imshow(cov_pct.values, aspect='auto', cmap=cmap, vmin=0, vmax=100)

ax.set_xticks(range(len(coverage.columns)))
ax.set_xticklabels(coverage.columns, fontsize=10)
ax.set_yticks(range(len(coverage.index)))
ax.set_yticklabels(coverage.index, fontsize=8)
ax.set_title('Data coverage (% of expected trading days per year)',
             fontsize=13, fontweight='bold', pad=12)
ax.set_xlabel('Year')
ax.set_ylabel('Stock')

# Add text in each cell

```

```

for i in range(len(coverage.index)):
    for j in range(len(coverage.columns)):
        days = coverage.values[i, j]
        pct = cov_pct.values[i, j]
        color = 'white' if pct < 30 or pct > 70 else 'black'
        if days > 0:
            ax.text(j, i, str(days), ha='center', va='center',
                    fontsize=6, color=color)

plt.colorbar(im, ax=ax, label='% of expected trading days', shrink=0.8)
plt.tight_layout()
plt.savefig('data_quality_coverage.png', dpi=120, bbox_inches='tight')
plt.show()
print('Green = good | Yellow = partial | Red = missing year')
print('Any RED cells for your 15 key stocks = check that year CSV file')

```

Cell 5 — RSI and return distributions

These histograms show whether your indicators look mathematically correct. RSI should form a bell curve centered around 40-60. Daily returns should form a narrow bell curve centered near 0.

```

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

df_ind_clean = df_ind.dropna(subset=['rsi_14', 'daily_return', 'ma_20'])

# RSI histogram
axes[0].hist(df_ind_clean['rsi_14'], bins=50, color='#1976d2', alpha=0.75,
            edgecolor='white')
axes[0].axvline(30, color='red', linestyle='--', linewidth=1.5, label='Oversold (30)')
axes[0].axvline(70, color='green', linestyle='--', linewidth=1.5, label='Overbought (70)')
axes[0].set_title('RSI distribution', fontweight='bold')
axes[0].set_xlabel('RSI value')
axes[0].legend(fontsize=8)
rsi_bad = ((df_ind_clean['rsi_14'] < 0) | (df_ind_clean['rsi_14'] > 100)).sum()
axes[0].text(0.05, 0.95, f'Out of range: {rsi_bad}',
            transform=axes[0].transAxes, fontsize=9,
            color='red' if rsi_bad > 0 else 'green',
            verticalalignment='top')

# Daily return histogram (zoom in to -10% to +10%)
ret_pct = df_ind_clean['daily_return'] * 100
axes[1].hist(ret_pct.clip(-10, 10), bins=60, color='#388e3c', alpha=0.75,
            edgecolor='white')
axes[1].axvline(0, color='black', linestyle='-', linewidth=1)
axes[1].set_title('Daily returns (clipped ±10%)', fontweight='bold')
axes[1].set_xlabel('Daily return (%)')
extreme_ret = (ret_pct.abs() > 50).sum()
axes[1].text(0.05, 0.95, f'Extreme (>50%): {extreme_ret}',
            transform=axes[1].transAxes, fontsize=9,
            color='red' if extreme_ret > 10 else 'green',
            verticalalignment='top')

# MA20 vs close_price scatter (they should be correlated)
merged = df.merge(df_ind_clean[['seance', 'isin_code', 'ma_20']],
                on=['seance', 'isin_code'], how='inner')
sample = merged.sample(min(3000, len(merged)), random_state=42)
axes[2].scatter(sample['close_p'], sample['ma_20'], alpha=0.15,
                s=5, color='#7b1fa2')

```

```

lims = [0, max(sample['close_p'].max(), sample['ma_20'].max()) * 1.05]
axes[2].plot(lims, lims, 'r--', linewidth=1, label='Perfect correlation')
axes[2].set_title('Close price vs MA20\n(should be on the diagonal)',
                  fontweight='bold')
axes[2].set_xlabel('Close price (DT)')
axes[2].set_ylabel('MA20 (DT)')
axes[2].legend(fontsize=8)

plt.tight_layout()
plt.savefig('data_quality_distributions.png', dpi=120, bbox_inches='tight')
plt.show()

# RSI check
print(f'RSI stats: min={df_ind_clean["rsi_14"].min():.1f} '
      f'max={df_ind_clean["rsi_14"].max():.1f} '
      f'mean={df_ind_clean["rsi_14"].mean():.1f}')
print('PASS: RSI in [0,100]' if rsi_bad == 0 else f'FAIL: {rsi_bad} RSI values out of
range')

```

Cell 6 — Final report: ready to train?

This cell runs all checks together and gives you a single GO or NO-GO answer.

```

print('=' * 55)
print('FINAL DATA QUALITY REPORT')
print('=' * 55)

results = []

# 1. Row counts
ok = len(df) >= 100_000
results.append(('Row count >= 100,000', ok, f'{len(df):,} rows'))

# 2. Stock count
ok = df['ticker'].nunique() >= 50
results.append(('Stock count >= 50', ok, f'{df["ticker"].nunique()} stocks'))

# 3. No NULLs in prices
price_null = df[['open_p', 'close_p', 'low_p', 'high_p']].isnull().sum().sum()
results.append(('No NULLs in prices', price_null == 0, f'{price_null} NULLs found'))

# 4. Price logic
df_c2 = df.dropna(subset=['open_p', 'close_p', 'low_p', 'high_p'])
bad_logic = (df_c2['high_p'] < df_c2['low_p']).sum()
results.append(('high >= low always', bad_logic == 0, f'{bad_logic} violations'))

# 5. RSI valid
df_i2 = df_ind.dropna(subset=['rsi_14'])
rsi_bad = ((df_i2['rsi_14'] < 0) | (df_i2['rsi_14'] > 100)).sum()
results.append(('RSI in [0, 100]', rsi_bad == 0, f'{rsi_bad} invalid values'))

# 6. Coverage join
conn2 = get_conn()
joined = pd.read_sql('''
    SELECT COUNT(*) AS n FROM daily_prices dp
    INNER JOIN computed_indicators ci
    ON dp.seance=ci.seance AND dp.isin_code=ci.isin_code

```

```

'''', conn2)
conn2.close()
join_rows = joined['n'].iloc[0]
ok = join_rows >= len(df) * 0.75
results.append(('Join coverage >= 75%', ok,
               f'{join_rows:,} rows ({join_rows/len(df)*100:.0f}% of prices)'))

# 7. Key stocks have enough history
key_stocks = ['AMEN BANK', 'BIAT', 'BNA', 'STB', 'ATB',
              'SFBT', 'ARTES', 'CARTHAGE CEMENT', 'UIB']
stock_days = df.groupby('ticker')['seance'].count()
missing_key = [s for s in key_stocks
               if stock_days.get(s, 0) < 500]
results.append(('Key stocks have 500+ days',
               len(missing_key) == 0,
               f'Missing: {missing_key}' if missing_key else 'All OK'))

# Print report
all_ok = True
for name, passed, detail in results:
    icon = 'PASS' if passed else 'FAIL'
    print(f' {icon} {name:<35} {detail}')
    if not passed: all_ok = False

print()
if all_ok:
    print('GO - data is clean. Proceed to feature_engineering.ipynb')
else:
    print('NO-GO - fix the FAIL items above before training')

```

Quick Fix Reference

If any check fails, here is the exact SQL to fix it in pgAdmin:

Problem	SQL Fix	Notes
high < low	DELETE FROM daily_prices WHERE plus_haut < plus_bas;	Remove the impossible rows entirely
close > high	UPDATE daily_prices SET cloture = plus_haut WHERE cloture > plus_haut * 1.001;	Cap close at the high price
close <= 0	DELETE FROM daily_prices WHERE cloture <= 0;	Cannot fix — remove the row
NULL in close_p	DELETE FROM daily_prices WHERE cloture IS NULL;	Cannot impute OHLCV safely
RSI out of [0,100]	DELETE FROM computed_indicators WHERE rsi_14 < 0 OR rsi_14 > 100;	Recompute indicators for that stock

ISIN format mismatch	UPDATE computed_indicators SET isin_code = TRIM(UPPER(isin_code));	Also run on daily_prices
Missing year for a stock	Re-run load_prices.ipynb with that year's file	Check data/prices folder for the file